TP CSS3 — Concevoir une interface web moderne et responsive

Niveau : 1ère année cycle ingénieur – informatique **Durée :** 3 heures **Prérequis :** HTML5 et notions de base de CSS3 **Approche :** pratique, guidée par problèmes **Projet fil rouge :** création progressive d'une interface de tableau de bord pour une application web.

1. Objectifs de l'atelier

À la fin du TP, l'étudiant doit être capable de :

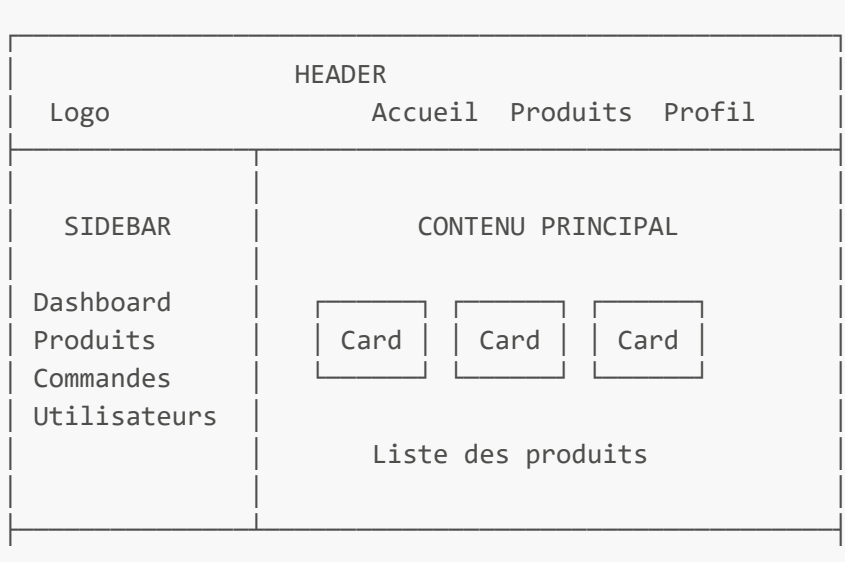
- structurer correctement une feuille CSS ;
- utiliser efficacement les sélecteurs CSS ;
- comprendre la cascade et la spécificité ;
- utiliser les unités CSS adaptées ;
- maîtriser le **Box Model** ;
- construire une mise en page avec **Flexbox** ;
- construire une mise en page avec **CSS Grid** ;
- utiliser les pseudo-classes et pseudo-éléments ;
- créer des interfaces responsive avec les media queries ;
- utiliser les variables CSS ;
- créer des transitions et animations simples ;
- organiser son CSS de manière maintenable.

L'objectif n'est donc pas de mémoriser les propriétés CSS, mais de savoir **choisir la bonne technique de mise en page et diagnostiquer un problème CSS**.

---

2. Projet fil rouge

Vous aller construire progressivement une interface :



```
│                    FOOTER                                │
└──────────────────────────────────────────────────────────┘
```

Au début, vous fournissez uniquement :

```
dashboard/
│
├── index.html
├── css/
│   └── style.css
└── images/
```

Vous allez construire le CSS progressivement.

Activité 1 — Structurer et cibler les éléments

Objectif

Réviser les sélecteurs mais surtout apprendre à les combiner proprement.

Vous fournissez :

```html
<header class="header">
    <h1>Dashboard</h1>

    <nav class="navigation">
        <a href="#">Accueil</a>
        <a href="#">Produits</a>
        <a href="#">Commandes</a>
    </nav>
</header>
```

Questions

Q1. Sélectionnez tous les liens du menu.

Q2. Sélectionnez uniquement les liens du `<nav>`.

Q3. Sélectionnez le premier lien.

Q4. Sélectionnez un lien lorsque la souris passe dessus.

Q5. Écrivez un sélecteur permettant de sélectionner uniquement les liens descendants de `.navigation`.

Puis introduire :

```
.navigation a
```

et :Les étudiants vont

```
.navigation > a
```

Question importante

> Quelle différence entre un descendant et un enfant direct ?

Activité 2 — Cascade, spécificité et héritage

Donnez volontairement :

```
p {
    color: blue;
}

.content p {
    color: green;
}

#message {
    color: red;
}
```

et :

```
<p id="message">Bienvenue</p>
```

Questions

1. Quelle couleur sera affichée ?
2. Pourquoi ?
3. Quelle règle possède la plus grande spécificité ?
4. Que se passe-t-il si on ajoute `!important` ?

Il faut comprendre progressivement :

```
!important
     ↓
```

```
Inline
     ↓
ID
     ↓
Classe / attribut / pseudo-classe
     ↓
Élément / pseudo-élément
```

Puis faire un exercice de **diagnostic CSS** où certaines propriétés apparaissent barrées dans les DevTools.

Activité 3 — Box Model et positionnement

On part d'une carte :

```html
<article class="card">
    <h2>Produit</h2>
    <p>Description du produit...</p>
    <button>Voir détails</button>
</article>
```

Vous devez construire :

```css
.card {
    width: 300px;
    padding: 20px;
    border: 1px solid #ddd;
    margin: 20px;
}
```

Puis utiliser les DevTools pour examiner le Box Model.

Questions

> Quelle est la largeur du contenu ?

> Quelle est la largeur réelle de l'élément ?

> Quelle différence entre `content-box` et `border-box` ?

Introduire :

```css
* {
    box-sizing: border-box;
}
```

Petit défi

Faire en sorte que trois cartes aient exactement la même largeur et la même hauteur.

Activité 4 — Flexbox

C'est l'une des parties centrales du TP.

On doit construire :

```
┌──────────────────────────────────────────────┐
│ Logo                 Accueil Produits Profil │
└──────────────────────────────────────────────┘
```

Avec :

```css
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

Questions guidées

Vous devez expérimenter :

```
display: flex;
```

puis :

```
justify-content
```

avec :

```
flex-start
center
flex-end
space-between
space-around
space-evenly
```

Puis :

```
align-items
```

et :

```
stretch
center
flex-start
flex-end
```

Exercice Flexbox 2

Créer :

```
┌──────────┐ ┌──────────┐ ┌──────────┐
│ Produit 1│ │ Produit 2│ │ Produit 3│
└──────────┘ └──────────┘ └──────────┘
```

avec :

```
.cards {
    display: flex;
    gap: 20px;
}
```

Puis demander :

> Que se passe-t-il lorsque la largeur disponible devient insuffisante ?

Introduire :

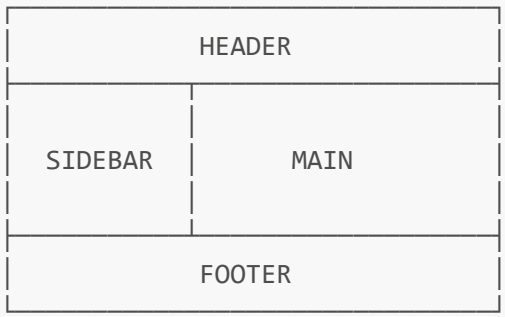
```
flex-wrap: wrap;
```

Puis :

```
flex: 1;
```

Activité 5 — CSS Grid

Ici, il faut construire la structure générale du dashboard.



On commence par :

```css
.dashboard {
    display: grid;

    grid-template-columns: 220px 1fr;

    grid-template-rows: 70px 1fr 50px;
}
```

Puis utiliser :

```css
header {
    grid-column: 1 / -1;
}
```

et :

```css
footer {
    grid-column: 1 / -1;
}
```

Questions

> Que signifie `1fr` ?

> Que signifie `1 / -1` ?

> Quelle différence entre Grid et Flexbox ?

Activité 6 — Responsive Design

À ce stade, l'application fonctionne sur desktop.

Mais elle doit également fonctionner sur :

```
Desktop
Tablet
Mobile
```

Vous devez identifier les problèmes.

Par exemple :

```
@media (max-width: 768px) {

    .dashboard {
        grid-template-columns: 1fr;
    }

    .sidebar {
        display: none;
    }

    .cards {
        flex-direction: column;
    }
}
```

Questions

> Que signifie `max-width: 768px` ?

> Quand cette règle est-elle appliquée ?

> Quelle différence entre `min-width` et `max-width` ?

Défi responsive

Faire passer :

```
DESKTOP

┌─────────┬──────────────────────┐
│         │ Card │ Card │ Card   │
│Sidebar  ├──────────────────────┤
```

```
|         |       Content          |
└─────────┴────────────────────────┘
```

à :

```
MOBILE

┌──────────────────┐
│      Header      │
├──────────────────┤
│       Card       │
├──────────────────┤
│       Card       │
├──────────────────┤
│       Card       │
├──────────────────┤
│      Content     │
└──────────────────┘
```

Activité 7 — CSS moderne

Cette activité permet d'introduire plusieurs mécanismes très utiles.

Variables CSS

Créer :

```css
:root {
    --primary-color: #2563eb;
    --secondary-color: #64748b;
    --background-color: #f8fafc;
    --spacing: 20px;
}
```

Puis :

```css
button {
    background-color: var(--primary-color);
}
```

Question

> Quel est l'intérêt des variables CSS dans un projet réel ?

Pseudo-classes

Créer des effets :

```
button:hover {
    ...
}
```

```
input:focus {
    ...
}
```

```
li:first-child {
    ...
}
```

Transitions

Ajouter :

```
button {
    transition: 0.3s;
}
```

Puis :

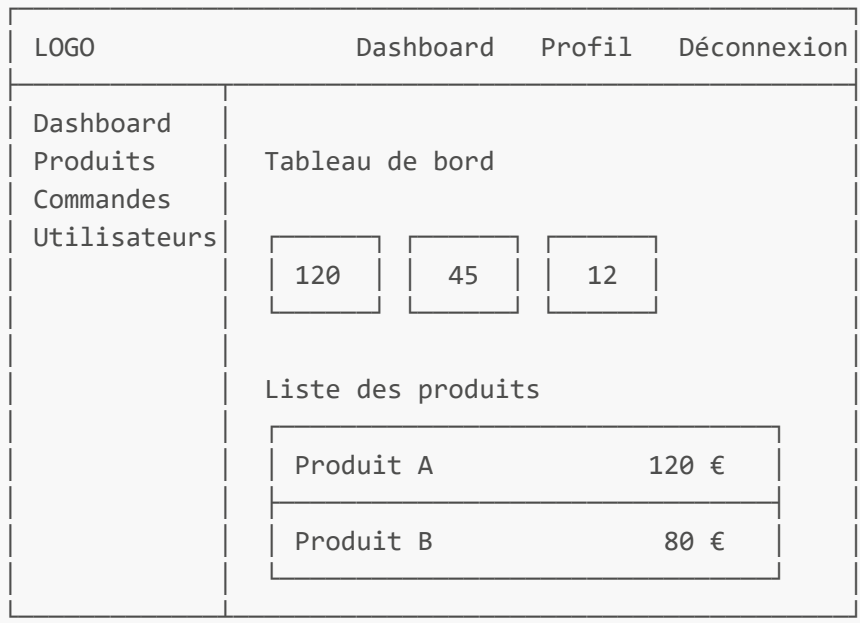
```
button:hover {
    transform: translateY(-2px);
}
```

Vous devez observer la différence entre un changement instantané et une transition.

Activité 8 — Défi final

À la fin du TP, je proposerais un vrai petit problème d'intégration.

Vous fournissez une **capture d'écran du dashboard final** :



Mission

> À partir du HTML fourni et de la capture cible, construisez entièrement la feuille CSS.

Contraintes :

- utiliser **Flexbox** pour le header et les composants internes ;
- utiliser **Grid** pour la structure principale ;
- utiliser des variables CSS ;
- utiliser au moins une media query ;
- utiliser `gap` plutôt que multiplier les marges ;
- rendre l'interface responsive ;
- ne pas utiliser Bootstrap ou une autre bibliothèque CSS.

Quand utiliser Flexbox ?

Pour organiser des éléments principalement sur **un axe** :

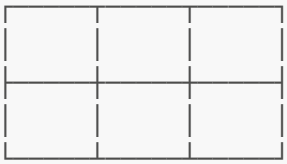
```
→ → → →
```

ou :

```
↓
↓
↓
```

Quand utiliser Grid ?

Pour une organisation **bidimensionnelle** :



Quand utiliser `position` ?

Pour des éléments nécessitant un positionnement particulier.

Quand utiliser les media queries ?

Pour adapter la présentation aux différentes tailles d'écran.

Quand utiliser les variables CSS ?

Pour centraliser les valeurs réutilisées et faciliter la maintenance.